

Problem 1 :

1. Wrong. Overfitting refers to situation where training error is close to zero while the testing error still large. In the double descent phenomenon, once the model passes the interpolation threshold, the training error is zero, but the testing error can decrease as model complexity increases.

2. Decrease. When $k=1$, the model relies on single nearest neighbor. So predictions are highly sensitive to noise in training data. When k increases, the model reduces the influence of any single noise, so the variance decreases but bias increases.

3.

$$q_j(x) = P(Y=j|X=x), \quad j \text{ is any label}$$

$$f^{\text{Bayes}}(x) = \arg \max_{j \in \text{all labels}} q_j(x)$$

$$P(Y \neq f^{\text{Bayes}}(x) | X=x) = 1 - \max_j q_j(x)$$

$$\begin{aligned} E(Y \neq f^{\text{Bayes}}(x)) &= \int (1 - \max_j q_j(x)) \cdot p(x) dx \\ &= 1 - \int \max_j q_j(x) \cdot p(x) dx \end{aligned}$$

4. Simple linear regression : $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$, $i=1, \dots, n$

$$\text{the fitted value } \hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$$

residual $e_i = y_i - \hat{y}_i = y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i$

Sum of residual $\sum_{i=1}^n e_i = \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)$

where $\hat{\beta}_0$ and $\hat{\beta}_1$ are parameter estimates obtained by minimizing the $\sum_{i=1}^n e_i$, $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$

Therefore,
$$\begin{aligned} \sum_{i=1}^n e_i &= \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i) \\ &= \sum_{i=1}^n (y_i - \bar{y} + \hat{\beta}_1 \bar{x} - \hat{\beta}_1 x_i) \end{aligned}$$

$$= \sum_{i=1}^n y_i - n\bar{y} + \hat{\beta}_1 n\bar{x} - \hat{\beta}_1 \sum_{i=1}^n x_i$$

Because $\bar{y} = \frac{\sum_{i=1}^n y_i}{n}$, $\bar{x} = \frac{\sum_{i=1}^n x_i}{n}$, $\sum_{i=1}^n y_i = n\bar{y}$, $\sum_{i=1}^n x_i = n\bar{x}$

$$= n\bar{y} - n\bar{y} + \hat{\beta}_1 n\bar{x} - \hat{\beta}_1 n\bar{x} = 0$$

Multiple Linear Regression :

$$y_i = \beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip} + \epsilon_i$$

fitted value $\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \dots + \hat{\beta}_p x_{ip}$

Now, we use matrix $y = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix}$ $X = \begin{bmatrix} x_{11} & \dots & x_{1p} \\ x_{21} & \dots & x_{2p} \\ \vdots & & \vdots \\ x_{n1} & \dots & x_{np} \end{bmatrix}$ $\hat{\beta} = \begin{bmatrix} \hat{\beta}_0 \\ \hat{\beta}_1 \\ \vdots \\ \hat{\beta}_n \end{bmatrix}$

residual $e = y_i - \hat{y}_i$ $\sum_{i=1}^n e_i = \sum_{i=1}^n y_i - \hat{y}_i$ it is equivalently written as

$$\|y - X\hat{\beta}\|$$

We know $\hat{\beta} = (X^T X)^{-1} X^T y$ obtained by zeroing the gradient.

$$\|y - X\hat{\beta}\| = \|y - X(X^T X)^{-1} X^T y\| = \|y - \hat{y}\| = 0$$

Therefore, $\sum_{i=1}^n e_i = 0$

5. We must use adjusted R square. Because we don't know the two datasets whether it's same or different.

We can use R square if two models have same parameters and same training data.

$$\text{Adjusted R square} = 1 - \frac{SSE / (n - p - 1)}{SST / (n - 1)}$$

$$\text{R square} = 1 - \frac{SSE}{SST}$$

If two models trained ^{on same dataset} have same parameters, the value p, n is same.

So Adjusted R square is just a rescaled version of

$1 - \frac{SSE}{SST} \frac{n-1}{n-p-1}$ depends on size of dataset and number of parameters

6. Linear Regression

{	Pros :	① needs less data
		② more interpretable.
{	Cons :	① inflexible

KNN

{	Pros :	① flexible
	Cons :	① need more data
		② less interpretable

Problem 2-1

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf

# load the dataset
data = pd.read_csv('/Users/mengchuishuo/Desktop/5054/HW1/Life Expectancy Data.csv')
```

```
In [2]: # Linear regression model : life expectancy ~ predicting variables except Country
data = data.dropna() #delete the NaN
X = data.drop(columns=["Life expectancy ", "Country"])
status_mapping = {"Developing": 1, "Developed": 2}
X["Status"] = X["Status"].map(status_mapping) #convert to numeric value
X = sm.add_constant(X) # Adds a constant term to the predictor
Y = data["Life expectancy "]

model = sm.OLS(Y, X).fit()
print(model.summary())
```

OLS Regression Results

Dep. Variable:	Life expectancy	R-squared:	0.839			
Model:	OLS	Adj. R-squared:	0.837			
Method:	Least Squares	F-statistic:	422.9			
Date:	Sun, 21 Sep 2025	Prob (F-statistic):	0.00			
Time:	13:06:32	Log-Likelihood:	-4421.2			
No. Observations:	1649	AIC:	8884.			
Df Residuals:	1628	BIC:	8998.			
Df Model:	20					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
const	307.2341	46.239	6.644	0.000	216.540	397.928
Year	-0.1272	0.023	-5.510	0.000	-0.172	-0.082
Status	0.8865	0.335	2.644	0.008	0.229	1.544
Adult Mortality	-0.0162	0.001	-17.171	0.000	-0.018	-0.014
infant deaths	0.0887	0.011	8.376	0.000	0.068	0.110
Alcohol	-0.1313	0.034	-3.901	0.000	-0.197	-0.065
percentage expenditure	0.0003	0.000	1.691	0.091	-4.83e-05	0.001
Hepatitis B	-0.0033	0.004	-0.732	0.464	-0.012	0.005
Measles	-1.033e-05	1.07e-05	-0.966	0.334	-3.13e-05	1.07e-05
BMI	0.0318	0.006	5.345	0.000	0.020	0.044
under-five deaths	-0.0666	0.008	-8.682	0.000	-0.082	-0.052
Polio	0.0058	0.005	1.132	0.258	-0.004	0.016
Total expenditure	0.0922	0.040	2.281	0.023	0.013	0.171
Diphtheria	0.0140	0.006	2.387	0.017	0.002	0.026
HIV/AIDS	-0.4481	0.018	-25.174	0.000	-0.483	-0.413
GDP	2.451e-05	2.83e-05	0.867	0.386	-3.09e-05	7.99e-05
Population	-6.085e-10	1.73e-09	-0.351	0.726	-4.01e-09	2.79e-09
thinness 1-19 years	-0.0058	0.053	-0.111	0.912	-0.109	0.097
thinness 5-9 years	-0.0501	0.052	-0.966	0.334	-0.152	0.052
Income composition of resources	10.4497	0.833	12.549	0.000	8.816	12.083
Schooling	0.8949	0.059	15.142	0.000	0.779	1.011
Omnibus:	31.845	Durbin-Watson:	0.707			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	58.052			
Skew:	-0.107	Prob(JB):	2.48e-13			
Kurtosis:	3.894	Cond. No.	3.80e+10			

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 3.8e+10. This might indicate that there are strong multicollinearity or other numerical problems.

According to the summary of model, observing the p-value. We can find Year, Status, Adult Mortality, infant deaths, Alcohol, BMI, under-five deaths, Total expenditure, Diphtheria, HIV/AIDS, Income composition of resources and Schooling are actually affecting the life expectancy.

Problem 2-2

Yes. According to the summary of model, observing the confidence interval. The coefficient of "Adult Mortality" and "HIV/AIDS" in 95% confidence interval are all negative. Adult Mortality is

from -0.018 to -0.014. HIV/AIDS is from -0.483 to -0.413. Therefore, these predictors have negative impact on the life expectancy.

Problem 2-3

```
In [3]: ci = model.conf_int(alpha=0.03).loc[["Schooling", "Alcohol"]]

alpha = 0.03 # construct confidence interval is 97%
lower_quantile = alpha/2
upper_quantile = 1 - alpha/2
ci.columns = [f"{lower_quantile:.3f}", f"{upper_quantile:.3f}"]
ci
```

```
Out[3]:
```

	0.015	0.985
Schooling	0.766518	1.023247
Alcohol	-0.204419	-0.058210

According to the confidence interval. The coefficient of "Schooling" in 97% confidence interval is positive from 0.766518 to 1.023247. The coefficient of "Alcohol" in 97% confidence interval is negative from -0.204419 to -0.058210. Therefore, Schooling has positive impact on the life expectancy. And Alcohol has negative impact on the life expectancy.

Problem 2-4

When multiple variables have a p - value of 0.000, the t - value is used to distinguish them. The larger the absolute value of the t - value, the stronger the explanatory power of the variable for the dependent variable. Therefore, the top-seven most influential predictors are HIV/AIDS, Adult Mortality, Schooling, Income composition of resources, under-five deaths, infant deaths, Year.

```
In [4]: X_smaller = data[[" HIV/AIDS", "Adult Mortality", "Schooling",
                        "Income composition of resources",
                        "under-five deaths ", "infant deaths", "Year"]] # choose the 7-top influential predictors

X_smaller = sm.add_constant(X_smaller)

Y_smaller = data["Life expectancy "]

model_smaller = sm.OLS(Y_smaller, X_smaller).fit()

print(model_smaller.summary())
```

Dep. Variable:	Life expectancy	R-squared:	0.824
Model:	OLS	Adj. R-squared:	0.823
Method:	Least Squares	F-statistic:	1096.
Date:	Sun, 21 Sep 2025	Prob (F-statistic):	0.00
Time:	13:06:32	Log-Likelihood:	-4493.3
No. Observations:	1649	AIC:	9003.
Df Residuals:	1641	BIC:	9046.
Df Model:	7		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	285.5374	45.502	6.275	0.000	196.289	374.786
HIV/AIDS	-0.4498	0.018	-24.615	0.000	-0.486	-0.414
Adult Mortality	-0.0181	0.001	-18.845	0.000	-0.020	-0.016
Schooling	1.0631	0.054	19.677	0.000	0.957	1.169
Income composition of resources	11.9119	0.825	14.440	0.000	10.294	13.530
under-five deaths	-0.0675	0.007	-9.171	0.000	-0.082	-0.053
infant deaths	0.0871	0.010	8.796	0.000	0.068	0.106
Year	-0.1158	0.023	-5.105	0.000	-0.160	-0.071
Omnibus:	35.229	Durbin-Watson:		0.657		
Prob(Omnibus):	0.000	Jarque-Bera (JB):		70.102		
Skew:	-0.081	Prob(JB):		5.99e-16		
Kurtosis:	3.997	Cond. No.		1.01e+06		

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.01e+06. This might indicate that there are strong multicollinearity or other numerical problems.

Problem 2-5

```
In [5]: new_obs = pd.DataFrame({
    "const": [1],
    "HIV/AIDS": [0.5],
    "Adult Mortality": [125],
    "Schooling": [18],
    "Income composition of resources": [0.9],
    "under-five deaths": [2],
    "infant deaths": [94],
    "Year": [2008]
})

pred = model_smaller.get_prediction(new_obs)

pred_summary = pred.summary_frame(alpha=0.01)
print(pred_summary)
```

	mean	mean_se	mean_ci_lower	mean_ci_upper	obs_ci_lower	obs_ci_upper
0	88.38095	0.926114	85.99266	90.76924	78.544814	98.217086

Problem 2-6

The AIC of full model is 8884, and the AIC of smaller model is 9003. The model with the smallest AIC is preferred. Therefore, the full model is better to fit the data. However, the smaller model is also attractive because it achieves the similar R^2 with few predictors, and smaller model is easy to interpret.

Problem 3

```
In [6]: # load the dataset firstly
train = pd.read_csv("/Users/mengchuishuo/Desktop/5054/HW1/boston_housing_train.csv")
test = pd.read_csv("/Users/mengchuishuo/Desktop/5054/HW1/boston_housing_test.csv")

X_train = train.drop(columns=["medv"]).values
y_train = train["medv"].values

X_test = test.drop(columns=["medv"]).values
y_test = test["medv"].values
```

```
In [7]: # construct KNN instead of directly using KNN in Python
class KNN:
    def knn_predict(self, X_train, y_train, X_test, K):
        y_pred = []
        for X in X_test:
            distances = np.sqrt(np.sum((X_train - X) ** 2, axis = 1))
            neighbors = np.argsort(distances)[:K]
            pred = np.mean(y_train[neighbors])
            y_pred.append(pred)
        return np.array(y_pred)

    def mse(self, y_true, y_pred):
        return np.mean((y_true - y_pred) ** 2)
```

Problem 3-1

```
In [8]: import time
import numpy as np
Ks = list(range(1,21))
knn = KNN()
results_no_std = []
for K in Ks:
    start = time.time()
    y_pred = knn.knn_predict(X_train, y_train, X_test, K)
    end = time.time()
    mse_val = knn.mse(y_test, y_pred)
    results_no_std.append((K, mse_val, end - start))
```

```
In [9]: results_no_std
```

```
Out[9]: [(1, 44.51708661417323, 0.007037162780761719),
(2, 46.05590551181103, 0.005452156066894531),
(3, 41.52323709536307, 0.005917072296142578),
(4, 40.88791830708661, 0.005707979202270508),
(5, 42.24112440944882, 0.005911111831665039),
(6, 43.889407261592304, 0.005583763122558594),
(7, 43.985068295034544, 0.007074832916259766),
(8, 42.83032357283464, 0.005741119384765625),
(9, 44.04344026441139, 0.005173921585083008),
(10, 45.6143283464567, 0.006474018096923828),
(11, 45.783520531007994, 0.005694866180419922),
(12, 45.87835465879264, 0.005568027496337891),
(13, 45.76500628989423, 0.0073070526123046875),
(14, 46.51385987465852, 0.005715370178222656),
(15, 46.5348738407699, 0.0074460506439208984),
(16, 48.18960999015748, 0.005529880523681641),
(17, 49.132718851320035, 0.005030155181884766),
(18, 48.92646058131623, 0.00485682487487793),
(19, 50.128822823739824, 0.0054149627685546875),
(20, 51.09004133858268, 0.005795717239379883)]
```

According to mse and time, we can find the best K value is 4. When K = 4, the model has the least mse and the shortest time.

Problem 3-2

```
In [10]: # standardize the predictor
mean = X_train.mean(axis=0)
std = X_train.std(axis=0)
X_train_std = (X_train - mean) / std
X_test_std = (X_test - mean) / std

results_std = []
for K in Ks:
    start = time.time()
    y_pred = knn.knn_predict(X_train_std, y_train, X_test_std, K)
    end = time.time()
    mse_val = knn.mse(y_test, y_pred)
    results_std.append((K, mse_val, end - start))
```

```
In [11]: results_std
```



```
Out[11]: [(1, 25.46929133858267, 0.00797891616821289),
(2, 16.777578740157477, 0.008079051971435547),
(3, 19.73487314085739, 0.005347251892089844),
(4, 20.019940944881885, 0.006626129150390625),
(5, 21.226831496062992, 0.005481243133544922),
(6, 22.255448381452318, 0.005274772644042969),
(7, 21.654661738711233, 0.005605936050415039),
(8, 20.868348917322834, 0.006175994873046875),
(9, 21.036780402449686, 0.00675511360168457),
(10, 20.50813543307087, 0.006303071975708008),
(11, 20.584192750699557, 0.005807161331176758),
(12, 20.443277559055122, 0.00639796257019043),
(13, 20.855489912873313, 0.005533933639526367),
(14, 21.566415715892653, 0.0064280033111572266),
(15, 21.72329658792652, 0.005079030990600586),
(16, 22.093315083661416, 0.005002021789550781),
(17, 22.66307250088548, 0.0049707889556884766),
(18, 23.372526489744338, 0.004981040954589844),
(19, 24.198553449516872, 0.004961729049682617),
(20, 24.45389448818897, 0.004987955093383789)]
```

According to mse and time, after standardize the predictor variables, we can find the best K value is 2. When K = 2, model has least mse and the short time.

Problem 3-3

Yes, standardization improve the performance. After standardization, the test mse descends from 40.88791830708661 to 16.777578740157477, we can see it drops significantly. While the running time remains nearly same. Therefore, I think standardization improve the performance.